

《软件架构模式》完整知识点整理

课件来源：南京大学软件学院 吕骏 助理研究员

整理范围：71页PPT全部知识点，按9个时代逐一拆解

关联考试：2026年《软件体系结构》期末考试（含C/S→分层→SOA演进论述题10分、事件驱动架构6分等）

🎯 这门课的核心目标不是记住术语，而是训练“看懂时代矛盾并做边界取舍”的能力。

目录

- 一、课程核心哲学（必背）
- 二、架构演进全景图
- 三、九个时代的逐一拆解
 - 第0代：最早期软件开发（1940s-1960s）
 - 第1代：主机/终端（1960s-1980s）
 - 第2代：C/S 架构（1980s-1990s）
 - 第3代：三层/分层（1990s-2000s）
 - 第4代：SOA（2000s）
 - 第5代：微服务（2010s）
 - 第6代：事件驱动/云原生（2015-2020s）
 - 第7代：AI增强（2023-至今）
 - 第8代：AI原生（2024-至今）
- 四、总结与选型指南
- 五、关联考试答题模板

一、课程核心哲学（必背）

1.1 为什么要“把架构放回时代里讲”？

课件开篇就定下了这门课的基调——这不是一门“背术语”的课：

“如果不理解‘旧模式为何失效’，就很难理解‘新模式为何成立’。”

四个核心认知：

认知	含义
架构不是凭空发明的	每一种架构都在回应当时最紧迫的技术与业务矛盾
“先进”是相对的	同一种模式在一个时代是解药，在另一个时代可能是负担
真正要学的是迁移逻辑	知道旧模式为什么失效，才知道新模式为什么出现
架构演进不是“替代史”	是“主要矛盾迁移史”

1.2 架构选择的本质

架构选择的本质，是在特定时代约束下，对质量属性做“有意识的优先级排序”。

关键认知：

- 质量属性不会同时最优。性能、可维护性、可扩展性、安全性、可部署性常常彼此拉扯。架构选择本质上是排序。
- 谁是当时的主矛盾，谁就决定系统优先保什么、允许牺牲什么。
- 时代变化会重排优先级。约束从算力、组织、流量一路迁移到语义与认知，取舍也跟着迁移。

1.3 这门课要训练的能力

不是"记住术语"，而是"看懂时代矛盾并做边界取舍"。

AI时代尤为如此——AI原生不是"微服务+LLM"，而是主要矛盾再次迁移后的新骨架。

二、架构演进全景图

2.1 发展主线：从算力稀缺到机器认知

算力稀缺 → 多用户共享 → 富交互 → 系统内分层 → 企业级协同
→ 团队自治发布 → 平台化韧性 → 语义增强 → 认知式执行

2.2 九个时代一句话概括

时代	时间	一句话核心矛盾
最早期	1940s-1960s	如何高效使用昂贵算力
主机/终端	1960s-1980s	如何让很多人共享同一套核心事务系统
C/S	1980s-1990s	如何把交互能力下放给桌面用户
三层/分层	1990s-2000s	如何统一入口并把系统内职责分清
SOA	2000s	如何跨系统复用业务能力并完成企业级整合
微服务	2010s	如何让团队按业务能力独立演进与发布
事件驱动/云原生	2015-2020s	如何在大规模分布式环境中保持韧性与自动化运行
AI增强	2023-至今	如何把语义理解与知识增强接入既有软件
AI原生	2024-至今	如何工程化管理机器认知、任务执行与非确定性

2.3 架构基本单元的变化

架构史不是一串名词，而是一部"系统基本单元不断变化"的历史。

程序 → 最小交付单元是一个任务程序
层 → 最小治理单元是职责分层
服务 → 最小演进单元变成业务能力
事件 → 最小协作单元从调用转向业务事实
模型/Agent → 最小认知单元变成能感知、推理、调用工具的执行体

关键洞察：当最小治理单元变化时，测试方式、平台能力和组织边界也会跟着变化。

三、九个时代的逐一拆解

每个时代按统一框架组织：①为什么出现（新约束+核心矛盾）→ ②上一代为什么失效 → ③结构骨架（边界/职责/协作）→ ④如何解决核心问题 → ⑤质量属性取舍 → ⑥选课系统案例

第0代：最早期软件开发（1940s–1960s）

时代背景

- 硬件极其昂贵，编程首先服务于"让机器跑起来"
- 应用以科学计算、军工计算、批处理报表为主，交互几乎不存在
- 软件与硬件、操作员流程、具体任务强绑定，复用性很弱
- 技术关键词：真空管/晶体管计算机、批处理作业、汇编与早期高级语言

默认架构目标

1. 先让机器稳定运行
2. 提高昂贵算力利用率
3. 把重复计算做成制度化流程

默认目标会决定：当代架构优先优化什么，又会忽略什么。

主要矛盾

计算资源极其稀缺，但任务正在从"算一次"走向"重复算、稳定算、规模算"。

维度	特征
资源	CPU/内存/存储昂贵，首先追求机器利用率
需求	任务固定，但结果必须稳定、可重复
工程	程序往往为一台机器、一项任务定制
组织	程序员、操作员、业务之间隔着很长链路

结构骨架

输入介质 → [程序 + 数据文件] → 批处理/作业调度 → 输出

- 程序与数据组成作业，统一送入系统执行
- 执行围绕批处理与调度组织，强调吞吐与稳定
- 交互极弱，输出通常在作业完成后统一返回

本质：先把"重复任务"写进程序，再把"执行秩序"写进流程。

第1代：主机/终端（1960s–1980s）

这一代第一次真正把"共享核心系统"做成了企业计算的常态。

① 为什么出现

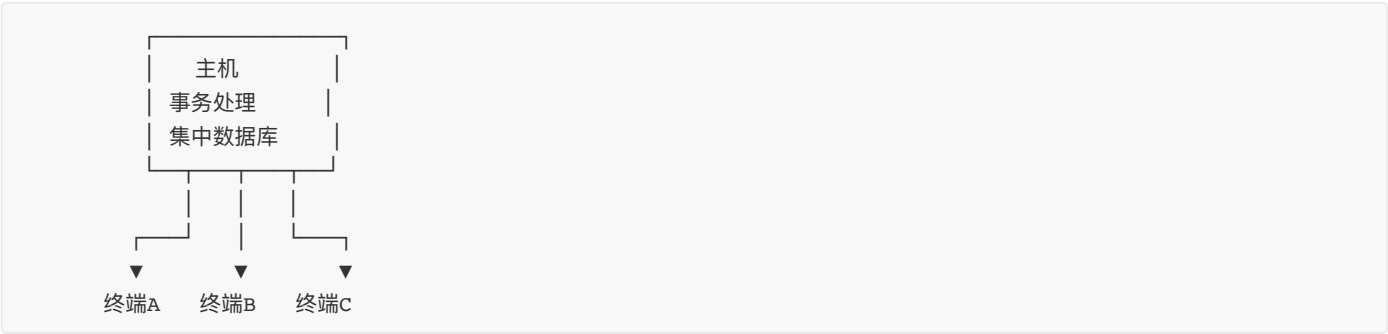
维度	内容
新约束	企业事务数字化
核心矛盾	越来越多用户需要共享同一套核心数据与事务能力
技术条件	大型主机成为中心计算平台，终端负责输入与显示
业务需求	银行、票务、工资、账务等重复事务开始在线化
默认目标	保证中心事务正确、安全，并让多终端共享访问
暂时忽略	丰富交互与部门级灵活定制仍让位于集中治理

② 上一代为什么失效

上一代：单机/批处理	新约束下的问题
擅长固定任务的稳定执行	无法支撑大量用户持续在线访问同一数据
强调昂贵算力利用率与结果可复现	缺少统一的事务、权限与审计控制
对离线作业和报表很有效	数据共享与并发访问一多，批处理思路就会失灵

→ 因此，系统必须从"执行任务"转向"服务一群用户"。

③ 结构骨架



- 主机承担几乎全部事务处理、数据存储与权限控制
- 终端主要负责输入、显示，不承载复杂业务逻辑
- 安全、审计和一致性都被集中到中心系统

本质：通过"强中心 + 弱终端"换取一致性、安全性与治理能力。

④ 如何解决核心问题

通过事务中心化，把"多人共享"从混乱访问变成可治理的制度化系统：

四大统一	内容
统一事务	同一份核心数据由中心系统负责一致性
统一安全	权限、审计与访问路径集中控制
统一资源共享	昂贵算力服务更多终端用户
统一运维治理	关键系统更容易被制度化治理

→ 解决的是"很多人如何共享核心事务系统"；但交互灵活性依然很弱。

⑤ 质量属性取舍

优先保障	相对牺牲
一致性	易用性
安全性	交互响应性
可靠性	局部可修改性
可审计性	

一句话取舍：中心化让事务和治理最稳，但也把交互贫瘠与变化成本一并锁进了中心。

⑥ 选课系统案例

教务主机（课程/名单/成绩）

- 院系办终端
- 教务处终端
- 财务处终端
- 学生查询终端

- 核心变化：从批处理名单到多终端共享教务数据
- 为什么这样设计：保证成绩、选课、容量一致
- 新问题：交互弱，变化都压在中心

第2代：C/S 架构（1980s-1990s）

PC革命把交互能力前移，软件第一次大规模贴近桌面用户。这一代第一次把"用户体验"作为企业软件可感知的目标。

① 为什么出现

维度	内容
新约束	PC与GUI普及
核心矛盾	用户开始要求更丰富、更快速的桌面交互
技术条件	PC、Windows、LAN普及，客户端开始具备本地算力
业务需求	软件深入财务、办公、制造、销售等桌面场景
默认目标	把交互带到桌面端，利用本地算力改善体验
暂时忽略	统一部署、版本治理与数据边界还没有成为首要问题

② 上一代为什么失效

上一代：主机/终端	新约束下的问题
把事务与数据集中管理得非常好	终端太"瘦"，难以支持复杂GUI与本地交互
强审计、强安全、强一致性	所有变化都压到中心系统，部门级快速建设很难
适合关键共享业务	PC算力已可用，完全不下放逻辑变得不经济

→ 因此，系统开始把"交互"前移到客户端，把"共享数据"留在服务器。

③ 结构骨架



- 客户端负责UI与部分本地逻辑，提升响应性与体验
- 服务器负责共享数据与部分公共逻辑
- 前后端通过网络协作，形成分布式桌面计算

本质：把"富交互"从中心系统前移到桌面端。胖客户端把交互做强，但也把部署复杂度带到了桌面端。

④ 如何解决核心问题

通过"客户端做交互、服务器保数据"的分工，第一次把企业软件真正贴近桌面用户：

四大改善	内容
界面前移	GUI与本地状态处理让体验显著改善
响应更快	一部分逻辑在本地完成，不必事事回中心
建设更快	部门级系统能较快上线，贴合局域网办公
共享仍存在	核心数据依旧保留在服务器端统一管理

→ 解决的是"怎样做富交互"；但它把部署和版本复杂度也搬到了客户端。

⑤ 质量属性取舍

优先保障	相对牺牲
易用性	可部署性
响应性	可维护性
交互体验	兼容性
	统一治理

一句话取舍：它用胖客户端换来了更好的体验，但也把安装、升级和版本控制问题带到了桌面端。

⑥ 选课系统案例



- **核心变化**：交互从中心系统前移到桌面客户端
- **为什么这样设计**：响应更快，界面更丰富
- **新问题**：安装、升级、兼容成本上升

第3代：三层/分层（1990s-2000s）

浏览器统一入口后，系统开始强调“服务器端分责”。这一代真正的进步不是“能上网了”，而是“系统内部开始更易维护”。

① 为什么出现

维度	内容
新约束	Web成为统一入口
核心矛盾	要降低部署复杂度，并把系统内部职责整理清楚
技术条件	浏览器普及，Web/App Server成为企业软件主流承载方式
业务需求	用户数扩大，统一入口与统一升级变得更加重要
默认目标	瘦客户端接入，服务端集中逻辑，内部按职责清晰分层
暂时忽略	系统之间如何协同，尚未成为最尖锐的问题

② 上一代为什么失效

上一代：C/S	新约束下的问题
桌面交互丰富，响应性强	客户端安装、升级、兼容成本随规模上升而爆炸
很适合局域网与部门级应用	客户端直接或强耦合访问数据层，不利于统一治理
能快速把软件贴近终端用户	一个系统里混杂了太多展示、业务和数据处理

→ 因此，重心从“前后分工”进一步转向“服务端内部清晰分层”。

③ 结构骨架



- **表示层**负责接入与界面呈现，统一用户入口

- **业务层**承载业务规则、流程控制与领域逻辑
- **数据层**负责持久化与数据访问，形成清晰职责边界

本质：通过职责分层控制变化影响，而不只是做"客户端瘦身"。

④ 如何解决核心问题

通过统一入口与职责分层，把"部署问题"和"维护问题"同时压了下来：

四大改进	内容
浏览器统一接入	客户端安装与升级成本明显下降
逻辑集中到服务端	业务规则更容易统一修改与治理
分层控制变化	界面变化、业务变化、数据变化尽量局部化
安全更清晰	数据访问不必暴露到大量客户端

→ 解决的是"系统内如何更清楚"；但**并没有解决"系统间如何协同"**。

⑤ 质量属性取舍

优先保障	相对牺牲
可维护性	极致性能
可修改性	跨系统协同灵活性
安全性	快速局部自治
可部署性	

一句话取舍：分层特别适合单系统治理，但复杂度仍然持续堆积在单应用边界之内。

★ **关键认知：**分层的关键价值不是"能上网"，而是把单系统内部职责理顺、把变化影响局部化。

⑥ 选课系统案例

学生/教师 → 浏览器 → 服务端分层

├— 表示层/Web

├— 业务层/选课规则

└— 数据层/课程成绩数据库

- **核心变化：**浏览器统一入口，服务端分层
- **为什么这样设计：**降低客户端维护成本
- **新问题：**复杂度仍堆在单应用内部

第4代：SOA（2000s）

★ **【考试重点】**这是2026年研究生考试 QII-2（10分论述题）的核心内容。

企业第一次系统性地把"能力复用"和"流程贯通"放到架构中心。

① 为什么出现

维度	内容
新约束	系统数量激增
核心矛盾	企业需要跨系统复用业务能力，并把端到端流程打通
技术条件	ERP、CRM、SCM、门户、支付等系统并存，异构成为常态
业务需求	同一条业务流程开始跨部门、跨系统、跨组织运行
默认目标	把系统功能提升为服务能力，通过契约与治理保持秩序
暂时忽略	快速独立发布与团队自治还没有压过企业级整合诉求

问题边界从"一个系统如何组织"转向"很多系统怎样协作"。

② 上一代为什么失效

上一代：三层/分层	新约束下的问题
很擅长把单个系统内部职责划清	系统之间依旧割裂，集成常靠大量点对点接口
统一入口与服务端治理都更容易	同一业务能力在不同系统中重复实现
对单应用维护性帮助很大	跨系统流程一变动，协调成本就会迅速上升

→ 因此，企业需要从"应用视角"上升到"服务能力视角"。

③ 结构骨架



- 能力通过服务契约暴露，不再只藏在某个应用内部
- ESB/中间件负责路由、转换、编排与集成
- 治理体系负责版本、规范、注册、权限与生命周期

本质：把"系统功能"提升为"可复用的服务能力"。

④ 如何解决核心问题

通过"契约 + 总线 + 编排 + 治理"，把企业从孤岛系统集成拉向了能力网络：

四大能力	内容
契约标准化	能力接口与系统内部实现脱钩
服务复用	订单、客户、支付等能力不必每个系统重写
流程编排	跨系统业务流程可以显式串起来
异构整合	不同技术栈也能进入统一企业流程

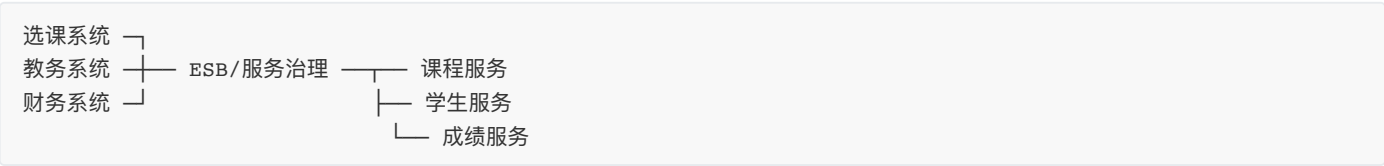
→ 解决的是"全企业协同"；代价是治理体系更重、中心中间件更强。

⑤ 质量属性取舍

优先保障	相对牺牲
互操作性	简单性
可复用性	响应性能
可组合性	轻量部署
可治理性	局部演进速度

一句话取舍：它解决了企业整合，但也让服务粒度、治理体系和ESB成为新的重量级约束。

⑥ 选课系统案例



- 核心变化：把课程、学生、成绩提升为可复用服务
- 为什么这样设计：跨系统的选课-缴费-成绩流程可以编排
- 新问题：服务粒度偏粗、中心总线成为瓶颈

第5代：微服务（2010s）

互联网业务节奏倒逼"按业务能力拆分 + 独立部署"。

① 为什么出现

维度	内容
新约束	交付频率大幅提升
核心矛盾	团队要更快迭代，但SOA的重治理与粗粒度服务拖慢了速度
技术条件	REST API、CI/CD、容器与DevOps实践逐渐成熟
业务需求	产品高频试错，团队希望围绕一块业务能力端到端负责
默认目标	按业务能力拆分，支持独立开发、独立部署、独立扩展
暂时忽略	分布式复杂度、调用链与运维成本往往被低估

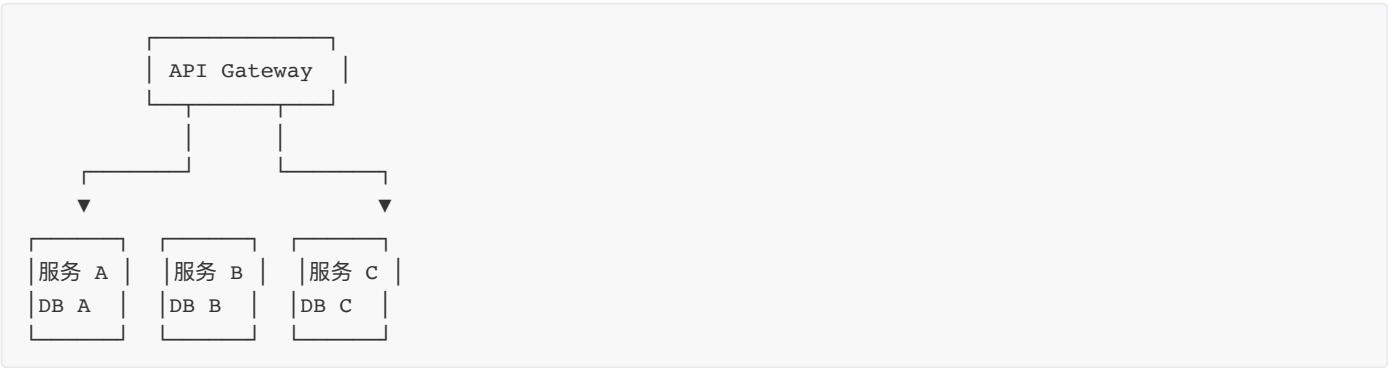
这一代的关键词不是"集成", 而是"演进速度"。

② 上一代为什么失效

上一代：SOA	新约束下的问题
很擅长企业级服务复用与流程编排	服务边界偏粗，不易与快速变化的产品边界对齐
可以让异构系统进入统一契约体系	中心治理与总线容易形成变更瓶颈
强治理适合大企业整合	跨团队依赖多，难以实现真正的独立发布

→ 因此，服务必须更贴近业务能力、团队边界和发布节奏。

③ 结构骨架



- 系统围绕相对独立的业务能力拆成多个小服务
- 每个服务由相对独立的团队负责其开发、部署与运维
- 接口协作替代共享实现，数据尽量跟随服务边界自治

本质：让"系统边界""团队边界""发布边界"尽量对齐。

④ 如何解决核心问题

通过更小的业务边界与独立部署，把交付链条显著缩短：

四大改进	内容
边界更小	单次改动影响范围更可控
团队自治	一支团队可以端到端拥有一块能力
部署更独立	不必每次都整体发版整套系统
扩展更局部	热点能力可以单独扩容

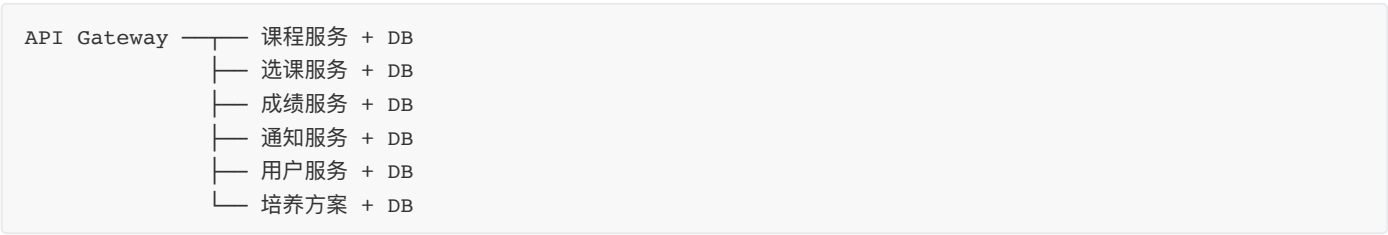
★ 关键认知：微服务不是"服务更多"，而是"演进单位更小、交付链更短"。

⑤ 质量属性取舍

优先保障	相对牺牲
可部署性	一致性
可扩展性	运维简单性
可修改性	故障定位性
演进速度	全局可理解性

一句话取舍：复杂度并没有消失，而是从代码内部转移到了分布式协作、数据一致性与运维层。

⑥ 选课系统案例



- 核心变化：围绕业务能力拆成多个小服务
- 为什么这样设计：局部改动影响更小
- 新问题：调用链、数据一致性、排障更难

第6代：事件驱动/云原生（2015-2020s）

★【考试重点】这是2026年研究生考试 QI-9（6分简答题）的直接考点。

系统不只要拆开，还要在大规模分布式环境中活得稳。

① 为什么出现

维度	内容
新约束	系统规模与流量波动持续放大
核心矛盾	同步链路过长、扩缩容困难、人工运维不可持续
技术条件	公有云、容器编排、流式平台、可观测性体系成熟
业务需求	系统要高弹性、自动恢复、按需扩缩容，并支持实时协同
默认目标	降低同步耦合，把运行能力下沉为平台能力
暂时忽略	端到端可理解性与时序复杂度常常会上升

复杂度开始从"开发期怎么拆"迁移到"运行期怎么活得稳"。

② 上一代为什么失效

上一代：微服务	新约束下的问题
边界小、发布快、团队自治强	全同步调用链会放大尾延迟和级联故障
适合业务高频演进	服务变多后，发布、扩容、恢复、观测仍高度依赖人工
更容易围绕业务能力扩展组织	运行能力如果不平台化，规模效应就无法形成

→ 因此，系统既要改变协作方式，也要改变运行方式。

③ 结构骨架



- 事件驱动：生产者发布事实，消费者按需订阅并处理
- 消息/流平台承担传递、缓冲、削峰与异步解耦
- 云原生平台承担编排、扩缩容、恢复、配置与观测

本质：事件驱动解决"如何协作"，云原生解决"如何运行"。

④ 如何解决核心问题

通过异步事件与平台化运行能力，让分布式系统更弹性、更韧性、更自动化：

四大能力	内容
异步解耦	生产者不必强依赖消费者实时在线
削峰与缓冲	流量波峰不再直接压垮下游
自动化运行	扩容、恢复、调度尽量交给平台处理
可观测性前置	运行状态必须被显式监控和追踪

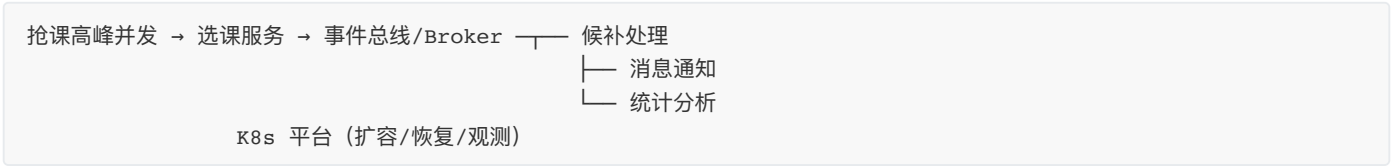
★ 关键认知：它把"系统运行能力"正式拉进了架构主体，而不再只是运维附属。

⑤ 质量属性取舍

优先保障	相对牺牲
弹性	可理解性
韧性	可调试性
可扩展性	时序可预测性
可用性	强一致性
自动化运维能力	

一句话取舍：它让系统更能扛、更会自恢复，但也让时序推理、排障和最终一致性治理更难。

⑥ 选课系统案例



- **核心变化**：从服务拆分走向异步协作与平台化运行
- **为什么这样设计**：削峰填谷，高峰期自动扩缩容
- **新问题**：时序更复杂，排障更依赖可观测性

第7代：AI增强（2023-至今）

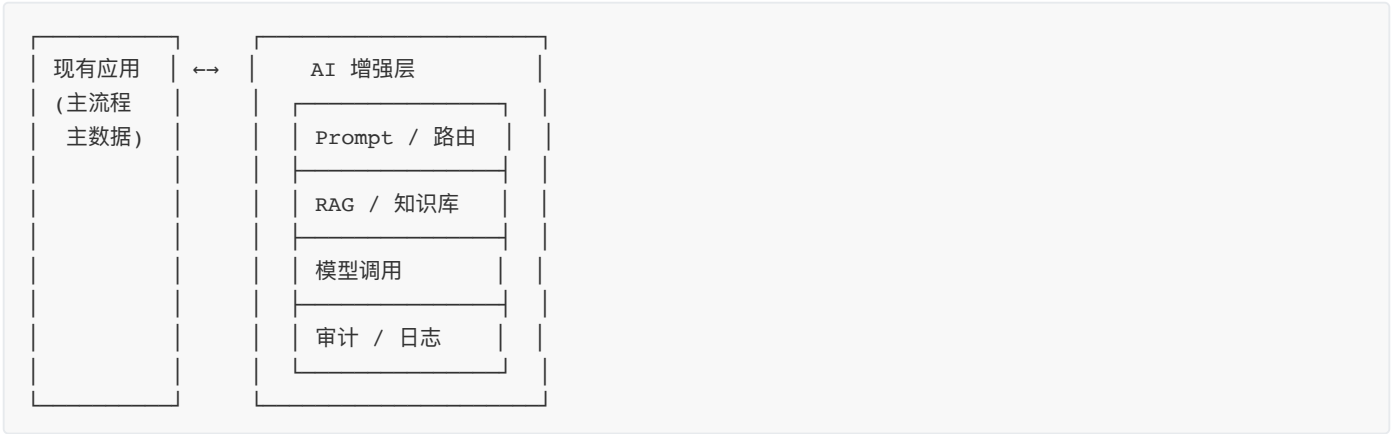
AI首先以外挂能力进入既有软件，而不是立刻重写一切。

① 为什么出现

维度	内容
新约束	用户开始直接用自然语言工作
核心矛盾	传统软件擅长规则与流程，却不擅长开放式语义任务
技术条件	LLM API、RAG、提示词编排与工具调用快速成熟
业务需求	企业希望先把AI接到现有系统上，而不是推翻旧系统
默认目标	给旧系统接入自然语言入口、知识增强与内容生成能力
暂时忽略	复杂长链路任务执行与非确定性治理还未成为主战场

AI在这一阶段更像"外挂能力层"，而不是系统运行时的中心。

② 结构骨架



- **现有业务系统**继续保留主流程与主数据
- **新增模型调用层**，负责prompt构造、路由与上下文组织
- **RAG、工具调用、审计与评估**成为常见新组件

本质：传统系统继续定义主流程，AI负责增能而不接管全流程。

③ 如何解决核心问题

通过"旧系统保主流程 + AI负责语义增能"，实现低风险接入：

四大能力	内容
自然语言入口	把复杂菜单和检索流程前移为对话入口
知识增强	RAG让模型能利用企业私域知识
工具接入	模型不仅能回答，还能查询和触发动作
治理补丁	日志、权限、评估与审计开始补上

★ 关键认知：这一阶段的关键不是"重写系统"，而是"在可控边界内先验证价值"。

④ 质量属性取舍

优先保障	相对牺牲
易用性	可预测性
语义适应性	可测试性
知识获取效率	成本稳定性
功能可扩展性	安全边界清晰性

一句话取舍：先追求"能增能、能试点"，而不是一开始就追求完全确定、完全自动化。

⑤ 选课系统案例

现有选课系统（主流程/主数据） ↔ AI增强层

Prompt / 权限校验

RAG / 培养方案知识库

模型调用

审计日志

学生提问："我还差哪些学分？"

→ AI负责解释、检索与生成，旧系统仍负责确定性流程

- 核心变化：在旧系统上接入自然语言与知识能力
- 为什么这样设计：低风险验证AI价值
- 新问题：输出不稳定，治理和评估更难

第8代：AI原生（2024-至今）

模型、Agent、工具、记忆与评测开始共同构成新的系统骨架。

① 为什么出现

维度	内容
新约束	AI不只回答问题，还要完成任务
核心矛盾	传统请求-响应式架构无法管理长链路、非确定性与多步协作
技术条件	Agent、工具使用、长上下文、记忆与评测框架快速出现
业务需求	企业开始期待AI规划、执行、回顾并与人协作完成任务
默认目标	把认知流程显式工程化，让非确定性系统变得可控
暂时忽略	若盲目上多Agent，复杂度会比价值增长得更快

真正的断点不是"有没有模型"，而是"模型是不是系统运行时的一部分"。

② 结构骨架



- **Agent/编排器**负责目标理解、任务规划与执行流程组织
- **工具、知识、状态、记忆层**成为模型可调用的运行时组件
- **评测、守护、权限与HITL**（Human-in-the-Loop）负责把非确定性纳入治理

本质：模型不再只是外部API，而是**系统运行时主体之一**。

③ 如何解决核心问题

通过把"上下文、工具、状态、评测、守护"显式工程化，来管理机器认知：

四大显式化	内容
显式状态	任务过程与中间结果不再隐含在单轮对话里
显式工具链	模型可以在边界内查询、执行、验证与回滚
显式评测	质量不只看可用性，还要看行为是否受控
显式守护	权限、成本、风险与人类接管点必须前置设计

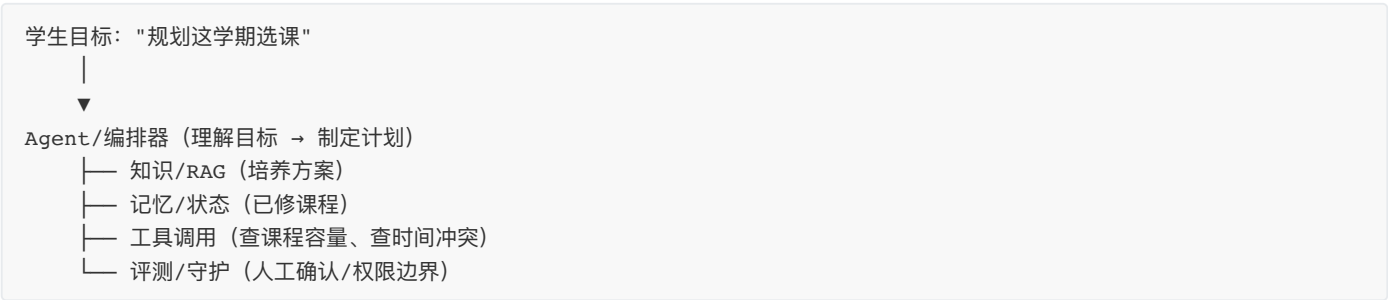
★ 关键认知：AI原生的真正挑战，不是"让模型更聪明"，而是"让系统更可控"。

④ 质量属性取舍

优先保障	相对牺牲
适应性	确定性
任务完成能力	可验证性
人机协作效率	成本可预测性
复杂 workflow 扩展能力	安全与责任边界清晰度

一句话取舍：真正难点不是"让模型更聪明"，而是"让非确定性系统在工程上更可控"。

⑤ 选课系统案例



- 核心变化：从问答助手走向学业规划Agent
- 为什么这样设计：支持多步规划、工具调用与反馈
- 新问题：权限、责任、成本和评测都更敏感

→ 这一代的选课系统，开始从"功能系统"走向"围绕认知过程组织的任务系统"。

四、总结与选型指南

4.1 架构演进全景总结

软件架构的历史，就是"在新的成本结构下，重画边界和协作方式"的历史。

最早期/主机前夜	→ 如何高效使用昂贵算力
主机/终端	→ 如何让很多人共享同一套核心事务系统
C/S	→ 如何把交互能力下放给桌面用户
三层/分层	→ 如何统一入口并把系统内职责分清
SOA	→ 如何跨系统复用业务能力并完成企业级整合
微服务	→ 如何让团队按业务能力独立演进与发布
事件驱动/云原生	→ 如何在大规模分布式环境中保持韧性与自动化运行
AI增强	→ 如何把语义理解与知识增强接入既有软件
AI原生	→ 如何工程化管理机器认知、任务执行与非确定性

4.2 架构基本单元的变化

时代	基本单元	变成什么
最早期	程序	最小交付单元是一个任务程序
分层	层	最小治理单元是职责分层
SOA/微服务	服务	最小演进单元变成业务能力
事件驱动	事件	最小协作单元从调用转向业务事实
AI原生	模型/Agent	最小认知单元变成能感知、推理、调用工具的执行体

4.3 今天如何选型：不是追新，而是识别你的主矛盾

你的情况	推荐架构
仍以确定性流程为主，团队规模不大	分层单体/模块化单体仍可能是最优解
业务域复杂、团队独立发布频繁	微服务可能合适，但必须有足够平台能力
强异步、高峰流量、实时协同	事件驱动与云原生平台优先考虑
想提升知识工作效率，但主流程仍旧确定	优先做AI增强，而非直接上多Agent
任务开放、需要多步推理与工具调用	再考虑AI原生架构与Agent编排

★判断顺序建议：先看需求确定性 → 再看组织边界 → 再看运行复杂度 → 最后才看"是不是时髦"。

4.4 从"管理计算资源"走向"管理机器认知"

如果说主机、C/S、微服务、云原生是在不断重画"计算边界"，那么AI原生正在迫使我们重画"认知边界、权限边界与责任边界"。

五、关联考试答题模板

5.1 QII-2：C/S → 分层/三层 → SOA 演进（10分论述题）

答题结构：

一、场景设定（1分） 选择一个具体系统场景（建议选"企业电商系统"或"选课系统"），说明该系统在不同时代面临的不同业务需求。 二、第一代 → 第二代：c/s → 三层/分层（3分） 1. c/s解决了什么？（富桌面交互、响应快、部门级快速上线） 2. c/s产生了什么问题？（部署噩梦、版本混乱、客户端直连数据） 3. 新约束是什么？（Web普及、用户规模扩大、需要统一入口） 4. 三层/分层怎么解决的？（浏览器统一接入、服务端分责、变化局部化） 5. 三层/分层遗留了什么？（系统间割裂、能力重复建设） 三、第二代 → 第三代：三层/分层 → SOA（3分） 1. 三层/分层解决了什么？（单系统内职责清晰、部署成本降低） 2. 三层/分层产生了什么问题？（系统间点对点集成、流程断裂） 3. 新约束是什么？（系统数量激增、异构常态化、跨系统流程需求） 4. SOA怎么解决的？（契约标准化、服务复用、ESB编排、异构整合） 5. SOA遗留了什么？（中心治理瓶颈、服务粒度偏粗、重协议）
--

四、质量属性权衡对比（2分）
用一张表对比三代架构优先保障和相对牺牲的质量属性。

五、总结（1分）
架构演进不是替代史，是主要矛盾迁移史。
每一次演进都是在新的约束下重排质量属性的优先级。

5.2 QI-9：事件驱动架构（6分简答题）

答题结构：

1. 基本思想（2分）
 - 组件通过发布和订阅"业务事实（事件）"进行异步协作
 - 不是同步的请求-响应调用
 - 生产者、消费者、事件总线/Broker三者角色
 - 事件使用过去时（"订单已创建"，不是"创建订单"）
2. 质量属性权衡（3分）

优先保障：弹性、韧性、可扩展性、可用性、自动化运维

相对牺牲：可理解性、可调试性、时序可预测性、强一致性
3. 一句话总结（1分）

用理解成本的上升换运行弹性的提升。

5.3 架构模式通用答题框架

课件中每一代架构都遵循同一套分析框架，考试时也可以按这个框架组织答案：

- ① 时代约束与核心矛盾（什么变了？）
- ② 上一代为什么失效（旧的为什么不行了？）
- ③ 结构骨架（边界、职责、协作怎么画？）
- ④ 如何解决核心问题（新架构做对了什么？）
- ⑤ 质量属性取舍（优先了什么，牺牲了什么？）
- ⑥ 遗留问题（这一代又留下了什么新问题？）

附录：九个时代质量属性取舍速查表

时代	优先保障	相对牺牲	一句话取舍
最早期	稳定性、算力利用率、可重复性	交互性、复用性、灵活性	先让机器稳定跑起来
主机/终端	一致性、安全性、可靠性、可审计性	易用性、交互响应性、局部可修改性	强中心+弱终端换一致性与治理
C/S	易用性、响应性、交互体验	可部署性、可维护性、兼容性、统一治理	胖客户端换体验，也带来部署负担
三层/分层	可维护性、可修改性、安全性、可部署性	极致性能、跨系统协同灵活性	适合单系统治理，复杂度堆在内部
SOA	互操作性、可复用性、可组合性、可治理性	简单性、响应性能、轻量部署、局部演进速度	解决企业整合，但治理体系变重
微服务	可部署性、可扩展性、可修改性、演进速度	一致性、运维简单性、故障定位性、可理解性	复杂度从代码转移到分布式协作
事件驱动/云原生	弹性、韧性、可扩展性、可用性、自动化运维	可理解性、可调试性、时序可预测性、强一致性	系统更能扛，但推理排障更难
AI增强	易用性、语义适应性、知识获取效率、功能可扩展性	可预测性、可测试性、成本稳定性、安全边界清晰性	先验证价值，再追求确定性
AI原生	适应性、任务完成能力、人机协作效率、复杂 workflow 扩展	确定性、可验证性、成本可预测性、安全责任边界	让非确定性系统在工程上更可控

 **推荐阅读路线：**架构基础（Bass, Clements, Kazman）→ 服务化/分布式（Newman, Kleppmann）→ AI系统工程（Chip Huyen）

 **关联复习：**本文档应与《软件体系结构-期末考试高分复习指南.md》配合使用，后者覆盖ADD设计方法、质量属性场景、设计决策、DDD聚合、视图文档化等全部考点。

整理自：南京大学软件学院《软件架构模式》课件（吕骏，2026）